Enterprise ELIS OS Audit Plan

https://gemini.google.com/app/cafd7b6db9a1c97b

User prompt: Act as a Principal Enterprise Software Architect, Senior Full-Stack Engineer, Database Architect, Application Security Engineer, Healthcare Interoperability Specialist, QA Architect, and DevOps Engineer. PROJECT NAME: NIKIL-LABCORE-ELIS-OS PRIMARY OBJECTIVE: Prepare a comprehensive, evidence-oriented audit plan for the existing project so that the next AI agent can inspect the actual repository, identify real bugs, validate existing features, discover architectural risks, and prepare a prioritized remediation plan. IMPORTANT: This is an existing project, not a greenfield project. Preserve working functionality. Do not assume that a documented feature is implemented or operational. Do not invent source files, test results, architecture details, compliance status, or defects. PHASE 1 — UNDERSTAND THE DOCUMENTATION Analyze the supplied project architecture documentation and extract the architecture it actually describes. Identify, where supported by the documentation: - Frontend frameworks and applications. - Backend services and APIs. - Database technology, ORM, schema, and migrations. - Authentication and authorization. - Patient and doctor management. - Test catalog and reference ranges. - Test orders and specimen tracking. - Result entry, validation, and approval. - Billing, GST, invoices, payments, and barcodes. - Laboratory reports and report delivery. - Analyzer middleware and ASTM/HL7 interfaces. - Inventory and reagent tracking. - Audit logging. - External integrations. - Automated testing. - Deployment infrastructure. - Environment configuration. If the documentation does not establish a detail, label it UNKNOWN or REQUIRES REPOSITORY VERIFICATION. PHASE 2 — BUILD A COMPLETE AUDIT CHECKLIST Create a detailed checklist for the next agent to inspect the actual repository. A. Repository and architecture - Application structure and entry points. - Duplicate implementations and legacy code. - Frontend/backend boundaries. - Service dependencies. - Configuration inconsistencies. - Error handling and logging. - Circular dependencies and unnecessary complexity. B. Frontend - Page and route completeness. - Forms, validation, and loading states. - Error and empty states. - API integration. - Authentication and session handling. - Authorization-aware UI. - Accessibility and responsive behavior. - Broken navigation and dead controls. C. Backend - API route completeness. - Input validation. - Consistent error responses. - Authentication and authorization enforcement. - Business-rule enforcement. - Transactions and concurrency. - Rate limiting where appropriate. - Idempotency for relevant operations. - Safe file uploads and report generation. - Logging without exposing sensitive data. D. Database - Schema consistency. - Relations, foreign keys, unique constraints, and indexes. - Prisma schema and migration alignment. - Transaction safety. - Data integrity. - Soft-delete behavior where implemented. - Race conditions and duplicate records. - Safe migration and rollback requirements. - Tenant isolation if multi-tenancy is implemented or planned. E. Laboratory workflows - Patient registration and search. - Doctor and referral mapping. - Test catalog and reference ranges. - Order creation. - Sample accession and barcode tracking. - Specimen collection and rejection. - Result entry and validation. - LOW, HIGH, and PANIC flags where applicable. - Pathologist approval. - Report generation and release. - Corrected reports and result history. - Critical-result notification and acknowledgement. - End-to-end patient-to-report workflow. F. Billing and business operations - Test pricing. - Discounts and tax calculations. - Invoice creation and numbering. - Payments, partial payments, balances, and refunds. - Barcode and invoice printing. - Duplicate billing prevention. - Inventory deduction and reagent lot tracking where implemented. - Financial auditability. G. Analyzer and interoperability - ASTM/HL7 parsing and validation where implemented. - Instrument-to-test mapping. - Patient and order matching. - Duplicate message handling. - Message acknowledgements. - Retry logic and failed-message

handling. - Raw message retention and safe replay. - Analyzer connectivity and monitoring. - Integration tests using synthetic messages. - Protection against malformed or malicious input. H. Security and privacy - Authentication and session security. - Role and permission enforcement. - Insecure direct object references. - Cross-user and cross-tenant access. - Secrets management. - Encryption requirements. - Audit log integrity. - Sensitive-data exposure in logs and errors. - Dependency vulnerabilities. - File and API access controls. - Backup security and recovery access. - Security testing requirements. I. Reliability and performance - Database connection handling. - Slow queries and missing indexes. - Resource leaks. - Timeouts and retries. - Queue processing where applicable. - Concurrent updates. - Health checks. - Observability. - Backup and restore. - Disaster recovery. - Graceful shutdown and startup behavior. J. Testing and delivery - Existing unit tests. - Integration tests. - End-to-end tests. - Test data safety. - Type checks and linting. - Production build. - CI/CD. - Environment configuration. - Migration testing. - Deployment and rollback documentation. PHASE 3 — PRIORITIZATION Define these priorities: P0: Critical security, data-loss, patient-safety, or core workflow blocker. P1: Major defect or high-impact failure. P2: Moderate defect, incomplete feature, or significant reliability gap. P3: Enhancement, optimization, or lower-priority improvement. Explain that severity must be assigned based on actual evidence and impact, not guesses. PHASE 4 — EVIDENCE REQUIREMENTS For every finding discovered later, require: - Unique issue ID. - Title. - Severity and priority. - Affected component. - Exact file path and relevant function or symbol. - Evidence. - Reproduction steps when possible. - Expected versus actual behavior. - Patient, business, security, or operational impact. - Root cause, if established. - Recommended fix. - Required regression test. - Dependencies and migration risks. - Verification status. Require the repository auditor to distinguish: 1. Confirmed defect. 2. Confirmed missing feature. 3. Incomplete implementation. 4. Potential risk requiring further verification. 5. Documentation-only claim. 6. Environment or dependency blocker. PHASE 5 — AUDIT EXECUTION PLAN Create a practical sequence for the repository auditor: 1. Inspect repository structure. 2. Inspect package manifests and scripts. 3. Inspect environment examples without exposing secrets. 4. Inspect database schema and migrations. 5. Map frontend pages to backend APIs. 6. Map critical business workflows. 7. Run safe, non-destructive static checks. 8. Run existing tests where feasible. 9. Reproduce suspected defects. 10. Record evidence. 11. Deduplicate findings. 12. Prepare a prioritized issue register. Do not prescribe destructive commands. Require approval before changing files, installing packages, running migrations, or modifying databases. PHASE 6 — REQUIRED OUTPUT Return these sections: 1. Documented Architecture Summary. 2. Confirmed Information versus Unknowns. 3. Complete Repository Audit Checklist. 4. Critical Workflow Test Matrix. 5. Security and Privacy Audit Matrix. 6. Database and Migration Audit Matrix. 7. Analyzer and Integration Audit Matrix. 8. Testing and Deployment Audit Matrix. 9. Issue Classification Rules. 10. Evidence Collection Template. 11. Recommended Audit Execution Order. 12. Final Checklist for Antigravity IDE. Do not implement code. Do not claim that the project is enterprise-ready, secure, compliant, or production-ready merely because the documentation describes these capabilities. Base architecture statements on the supplied documentation. Mark all code-level conclusions as requiring actual repository verification.

Response: 1. Documented Architecture Summary

The **NIKIL-LABCORE-ELIS-OS** (Version 2.5.0-PRO) architecture documentation describes an India-first diagnostic Laboratory Information System (LIS) targeting pathology laboratories, multi-branch hospital diagnostic networks, and NABL-accredited clinical facilities. All architectural claims below reflect the supplied documentation and archive file manifest; **every code-level implementation detail, runtime behavior, and compliance status requires actual repository verification.**

Architectural Domain	Documented Specification (NIKIL_LABCORE_ELIS_OS_COMPLETE_ARCHITECTURE.md)	Repository File Manifest Observations (REQUIRES REPOSITORY VERIFICATION)
Frontend Frameworks & Applications	• **Clinical Web App (frontend/apps/web/)**: Next.js 15.5.25 (App Router), React 18.3.1, TailwindCSS 3.4.19, Recharts 2.15.4, Lucide React 1.34.0, jsPDF 2.5.2, html2canvas, pdf-lib, qrcode 1.5.4 on Port 3001. • **Auth Portal (login-portal/)**: Vite 8.3.0, React 19.2.8, Zustand 5.0.15, React Router DOM 7.18.4, oxlint 1.81.0 on Port 5173. • **Legacy Frontend**: Mentioned as a 3rd frontend app in summary metrics.	• frontend/apps/web/src/app/ contains 24 route directories. • login-portal/src/pages/ contains 5 auth/dashboard pages. • Legacy/parallel UI code exists in public/ (index.html, app.js, aiStudio.js), frontend/src/components/PatientRegistration/, src/frontend/ (DoctorModuleApp.tsx, EMR/CPOE components), and client/package-lock.json.
Backend Services & APIs	• **Primary Backend (backend/api/)**: Node.js v20+/v22+, Express 5.2.1, TypeScript 5.9.3, Socket.io 4.7.5 on Port 5000 (/api/v1, /api/*) with 28 domain modules and 200+ REST endpoints. • **AI Bridge Services**: Python 3.10+, FastAPI, Uvicorn, httpx (gemini_spark_mcp_server.py on Port 8000 and gemini_web2api.py on Port 8001).	• backend/api/src/modules/ has 27 module folders (whatsapp-advanced files reside inside communications/; receipts/ has receipt.controller.ts, receipt.service.ts, receipt.validation.ts, but no receipt.routes.ts in the manifest). • Duplicate/legacy backend trees exist in `backend/api/src/controllers
Database, ORM, Schema &	• **Database**: PostgreSQL	•

Architectural Domain	Documented Specification (NIKIL_LABCORE_ELIS_OS_COMPLETE_ARCHITECTURE.md)	Repository File Manifest Observations (REQUIRES REPOSITORY VERIFICATION)
Migrations	14+/15.x/16.x (labcore_elis on Port 5432). • **ORM**: Prisma 7.9.1 with @prisma/adapter-pg and pg 8.23.0 pool. • **Schema**: 81 Prisma models (2,876 lines) across 8 domains. • **Risk Noted**: Schema drift risk on accounting_sequences table.	backend/prisma/schema.prisma and backend/prisma.config.ts exist. • 17 migration folders exist under backend/prisma/migrations/ (20260825092845_init to 20261014050000_enterprise_empi_module). • Ad-hoc SQL/schema scripts exist (backend/add-missing-column.js, backend/scripts/push-schema.ts, backend/scripts/_apply_sql.js).
Authentication & Authorization	• **Auth**: HMAC-SHA256 pepper + bcryptjs (cost 12), TOTP MFA (speakeasy/RFC 6238) with 10 backup codes, JWT Access (15m) + Refresh (7d) tokens, 5-attempt lockout, password history (last 5), device fingerprinting. • **RBAC**: 8 roles (SUPER_ADMIN, ADMIN, LAB_MANAGER, PATHOLOGIST, LAB_TECHNICIAN, PHLEBOTOMIST, RECEPTIONIST,	• Auth modules exist in backend/api/src/modules/auth/, backend/api/src/modules/users/auth.controller.ts, backend/api/src/controllers/auth.controller.ts, and src/middlewares/authMiddleware.js. • Multiple middleware files exist (auth.middleware.ts, auth.ts, rbac.middleware.ts).

Architectural Domain	Documented Specification (NIKIL_LABCORE_ELIS_OS_COMPLETE_ARCHITECTURE.md)	Repository File Manifest Observations (REQUIRES REPOSITORY VERIFICATION)
	BILLING_STAFF).	• Documented risk: password hash alignment issues across provisioning scripts.
Patient & Doctor Management	• **Patients (patients)**: UHID-YYYY-XXXXX sequence, EMPI duplicate detection (phone, government ID, email, Soundex phonetic scoring) and merge, draft registration, DPDP consent, PII masking. • **Doctors (doctors)**: Registry (REFERRING_DOCTOR, PATHOLOGIST, CONSULTANT), organizations, commission ledger (PERCENTAGE/FLAT_AMOUNT), payouts with TDS, digital signature governance, document vault.	• backend/api/src/modules/patients/ includes patient.service.ts, patient.empi.service.ts, patient.advanced.service.ts, patient.draft.service.ts. • backend/api/src/modules/doctors/ includes doctor.service.ts and doctor.commission.ts. • Legacy doctor/EMR files also exist in src/controllers/doctorController.js and src/models/Doctor.js.
Test Catalog & Reference Ranges	• **Catalog (tests)**: Test master with LOINC codes, sample types, analyte parameters (test_parameters), age/gender-segmented reference_ranges (newborn to senior), test_packages with auto-expansion, pricing tiers.	• backend/api/src/modules/tests/ has test.service.ts and test.advanced.service.ts. • Frontend has /tests, /tests/categories, /tests/packages,

Architectural Domain	Documented Specification (NIKIL_LABCORE_ELIS_OS_COMPLETE_ARCHITECTURE.md)	Repository File Manifest Observations (REQUIRES REPOSITORY VERIFICATION)
		/tests/parameters, /tests/reference-ranges, /tests/training.
Test Orders & Specimen Tracking	• **Orders (orders)**: ORD-YYYYMMDD-XXXXX, package auto-expansion, test deduplication, priority flags (URGENT, STAT, HOLD), atomic cancellation voiding invoices/payments. • **Specimens (samples)**: SMP-YYYYMMDD-XXXXX, 7-stage state machine (REGISTERED → COLLECTED → RECEIVED → PROCESSING → RESULTS_ENTERED → APPROVED → DISPATCHED), vacutainer classification, sample_tracking_history.	• backend/api/src/modules/orders/ and backend/api/src/modules/samples/ exist alongside duplicate backend/api/src/controllers/order.controller.ts. • Frontend includes specimen collection sheets, labels, redraw register, storage rack, and HIL quality checker components.
Result Entry, Validation & Approval	• **Results (results)**: Numeric/text/qualitative entry, unit validation, auto age/gender reference range flagging (NORMAL, ABNORMAL, CRITICAL, QC_FAILED), panic matrix (HH/LL), CriticalValueAcknowledgment, TAT tracking, pre-sign-off ResultAmendment. • **Approvals (approvals)**: Panic-prioritized queue, pathologist sign-off, reject	• backend/api/src/modules/results/ and backend/api/src/modules/approvals/ exist alongside duplicate backend/api/src/controllers/result.controller.ts. • Frontend has /results and /approvals routes plus src/frontend/components/PathologistVerification/.

Architectural Domain	Documented Specification (NIKIL_LABCORE_ELIS_OS_COMPLETE_ARCHITECTURE.md)	Repository File Manifest Observations (REQUIRES REPOSITORY VERIFICATION)
	flow. Roadmap lists Phase 4 as **40% in progress** (4.2 digital signature/seal and 4.3 Levey-Jennings/batch approval marked planned/incomplete).	
Billing, GST, Invoices, Payments & Barcodes	• **Invoices (invoices)**: INV-YYYYMMDD-XXXXX, intra-state CGST 9% + SGST 9% / inter-state IGST 18%, Indian banker's rounding, tax-inclusive/exclusive, zero-rated exemptions. • **Payments & POS**: payments (x-idempotency-key in-memory 24h TTL), cash-counter (single open shift per user, movements, daybook reconciliation), advances (wallet), refunds (approval chain), settlements, receivables, receipts (80mm ESC/POS), barcode (Code 128 Mod-103 50×25mm).	• Modules exist for invoices, payments, cash-counter, advances, refunds, settlements, receivables, receipts, barcode. • Frontend has multiple overlapping invoice components (LabCoreInvoice, LuxuryGSTInvoice, PixelPerfectInvoice, PremiumGSTInvoice, ProfessionalGSTInvoice, RefinedGSTInvoice).
Laboratory Reports & Delivery	• **Reports (reports)**: Server-side Puppeteer A4 NABL PDF, LOINC codes, QR code tamper verification (/verify-report/:id), time-limited report_share_links, versioned report_addendums.	• backend/api/src/modules/reports/ contains report.controller.ts, report.pdf.service.ts, report.routes.ts, report.service.ts, report.validation.ts.

Architectural Domain	Documented Specification (NIKIL_LABCORE_ELIS_OS_COMPLETE_ARCHITECTURE.md)	Repository File Manifest Observations (REQUIRES REPOSITORY VERIFICATION)
	• **Conflict Noted**: Roadmap lists Phase 5 (NABL ISO 15189 Puppeteer PDF Engine) at **0% Planned**, while Section 5.1 describes it as a module capability.	• Frontend also contains client-side PDF libraries (jsPDF, html2canvas, pdf-lib) and src/lib/pdf-operations.ts.
Analyzer Middleware & ASTM/HL7 Interfaces	• **Analyzers (analyzers)**: ASTM E1381/E1394 (`H	P
Inventory & Reagent Tracking	• Lot number tracking and expiry alerts are briefly mentioned under analyzers calibration. No standalone inventory or reagent backend module is listed among the 28 backend modules or 81 Prisma models.	• Frontend contains RealWorldReagentTracker.tsx and ReagentLotCorrelationSuite.tsx under frontend/apps/web/src/components/analyzers/. Backend persistence and API support are UNKNOWN / REQUIRES REPOSITORY VERIFICATION.
Audit Logging	• **Audit (audit)**: Logs actor (userId, role, IP), action, entity type/ID, JSON before/after delta, UTC timestamp, 7-year retention, and hash chaining for tamper detection.	• backend/api/src/modules/audit/ exists. Actual coverage across all mutating/PII-reading endpoints and hash-chain verification logic REQUIRES REPOSITORY VERIFICATION.
External Integrations	• WhatsApp Cloud API, SMS Gateway, SMTP/Gmail (nodemailer), ABDM Gateway (/v0.5/* RSA SPKI webhooks, M1/M2/M3), Government ID OTP service, Gemini AI (via Python FastAPI bridges), ZPL/PNG label printers, ESC/POS thermal printers.	• Roadmap lists Phase 6 (ABDM/DPDP) at 50% partial, Phase 7 (WhatsApp) at 40% partial, Phase 8 (AI Studio) at 40% partial, and Risk #4 notes /api/communications/{logs,stats,sms/templates} return 404.

Architectural Domain	Documented Specification (NIKIL_LABCORE_ELIS_OS_COMPLETE_ARCHITECTURE.md)	Repository File Manifest Observations (REQUIRES REPOSITORY VERIFICATION)
Automated Testing	• Not formally detailed in the architecture document (UNKNOWN).	• Root test/aiStudio.test.js, frontend/apps/web/src/components/invoices/PixelPerfectInvoice.test.tsx, and ad-hoc scripts in `backend/test-*.js
Deployment Infrastructure & Environment Config	• Recommended Nginx reverse proxy + PM2 cluster for Node/Next.js + Uvicorn for Python + PostgreSQL 14+ on private network. • 17+ environment variables documented across backend/.env, frontend/.env, and root/.env.	• DEPLOYMENT_GUIDE.md, backend/.env.example, frontend/apps/web/.env.example, and login-portal/.env.example exist. No Dockerfile, docker-compose.yml, or .github/workflows/ CI files appear in the file listing.

2. Confirmed Information versus Unknowns

The auditor must strictly separate what is **documented**, what is **observable in the file tree**, and what **requires code-level verification**.

Category	Confirmed by Documentation & File Manifest ([cite: 1, 2])	Contradictions / Ambiguities in Documentation & Tree ([cite: 1, 2])	UNKNOWN or REQUIRES REPOSITORY VERIFICATION
Backend Runtime & Entry Points	backend/api/src/server.ts and backend/api/src/app.ts exist alongside 27 module folders under backend/api/src/modules/.	ADR-001 and Risk #1 claim root index.js (Express 4 + Mongoose) is retired, yet index.js, src/controllers/*.js, src/models/*.js, src/routes/*.js, and public/ remain in the	• Which routes are actually mounted in backend/api/src/app.ts? • Are `backend/api/src/con

Category	Confirmed by Documentation & File Manifest ([cite: 1, 2])	Contradictions / Ambiguities in Documentation & Tree ([cite: 1, 2])	UNKNOWN or REQUIRES REPOSITORY VERIFICATION
		repository root.	trollers
Frontend Applications & Auth Flow	Both frontend/apps/web/ (Next.js) and login-portal/ (Vite) exist in the repo.	login-portal/ is documented as the standalone auth gateway on Port 5173, but frontend/apps/web/src/app/ ALSO contains /login, /mfa-setup, /mfa-verify, and /forgot-password routes.	• Which login UI is actually used in production/dev? • How are JWT tokens/cookies shared across ports 5173, 3001, and 5000? • Are src/frontend/ and frontend/src/ orphaned legacy code?
Database Schema & Migrations	backend/prisma/schema.prisma and 17 migration directories exist.	Risk #2 warns of schema drift on accounting_sequences, and backend/add-missing-column.js, backend/scripts/_apply_sql.js, and backend/scripts/push-schema.ts suggest manual schema mutations outside Prisma Migrate.	• Does schema.prisma actually contain 81 models, and does it compile cleanly? • Do the 17 SQL migrations match schema.prisma without drift? • Are destructive scripts (wipe-to-single-admin.ts, reset-all-data.ts) guarded against

Category	Confirmed by Documentation & File Manifest ([cite: 1, 2])	Contradictions / Ambiguities in Documentation & Tree ([cite: 1, 2])	UNKNOWN or REQUIRES REPOSITORY VERIFICATION
			production execution?
Clinical Workflows (Phase 1–5)	Phases 1–3 are marked 100% complete in the roadmap; Phase 4 is 40% in progress; Phase 5 is 0% planned.	Section 5.1 describes digital signatures, NABL doctor seals (Phase 4.2), and Puppeteer NABL PDF generation (Phase 5) as existing module features, contradicting Section 14's roadmap table.	• Does report.pdf.service.ts actually generate NABL PDFs via Puppeteer, or is it a stub? • Is the 7-stage sample state machine strictly enforced on the backend, or can statuses be skipped? • Are Westgard QC and 72h delta checks executed automatically on analyzer/manual result entry?
Financial & POS Operations	Modules for invoices, payments, cash-counter, advances, refunds, settlements, receivables, and receipts exist.	Payment idempotency is documented as an "in-memory store, 24h TTL", which is lost on server restart and unsafe across PM2 cluster workers.	• Are GST calculations, discounts, partial payments, wallet deductions, and order-cancel reversals wrapped in ACID prisma.$transaction blocks? • Which of the 6+

Category	Confirmed by Documentation & File Manifest ([cite: 1, 2])	Contradictions / Ambiguities in Documentation & Tree ([cite: 1, 2])	UNKNOWN or REQUIRES REPOSITORY VERIFICATION
			frontend invoice components is wired to /invoices?
Analyzer & External Integrations	modules/analyzers/, modules/abdm/, modules/communications/, and Python Gemini scripts exist.	Phases 6, 7, and 8 are marked 40–50% partial, and Risk #4 notes /api/communications/{logs,stats,sms/templates} return 404.	• Is there a real TCP/Serial ASTM or MLLP HL7 listener, or only an HTTP simulation endpoint? • Are ABDM M1/M2/M3 flows calling live/sandbox ABDM endpoints or returning mocked responses? • Are reagent/inventory UI components backed by real database tables or static demo data (realWorldDemoAnalyzers.ts)?
Security & Privacy	Security utilities (encryption.ts, field-crypto.ts, password.ts, totp.ts, jwt.ts) exist.	Risk #6 explicitly states government ID fields are stored in plaintext in some fields; Risk #3 notes password hashing mismatches in scripts; Risk #8 notes WebSocket auth is undocumented.	• Which PII fields are actually encrypted at rest? • Are RBAC middleware checks applied to every route in backend/api/src/mod

Category	Confirmed by Documentation & File Manifest ([cite: 1, 2])	Contradictions / Ambiguities in Documentation & Tree ([cite: 1, 2])	UNKNOWN or REQUIRES REPOSITORY VERIFICATION
			ules/*? • Are /verify-report/:id and shared report links leaking unmasked patient PII?

3. Complete Repository Audit Checklist

The repository auditor must systematically inspect the codebase against the following 10 audit domains (**A through J**). Every item must be verified directly in source code.

A. Repository and Architecture

- [ ] **Application Structure & Entry Points**: Verify the active entry points in package.json, backend/package.json, frontend/package.json, frontend/apps/web/package.json, and login-portal/package.json. Confirm how backend/api/src/server.ts and backend/api/src/app.ts bootstrap Express and Socket.io.
- [ ] **Duplicate Implementations & Legacy Code**:
  - [ ] Inspect root index.js, src/controllers/, src/models/, src/routes/, src/services/ai/, and public/ to confirm whether the legacy Express/Mongoose/vanilla-JS stack is completely dead or still invoked by any script or proxy.
  - [ ] Compare backend/api/middleware/ (auth.middleware.ts, auth.ts, error.middleware.ts, errorHandler.ts, validate.middleware.ts, validate.ts) with any middleware inside backend/api/src/ to identify which files are imported by app.ts and module routes.
  - [ ] Compare backend/api/src/controllers/ (auth.controller.ts, order.controller.ts, result.controller.ts), backend/api/src/routes/, and backend/api/src/schemas/ against backend/api/src/modules/{auth,orders,results}/ to detect shadowed or conflicting routes.
  - [ ] Compare backend/api/src/modules/users/auth.controller.ts against backend/api/src/modules/auth/auth.controller.ts.
  - [ ] Inspect duplicate frontend trees (frontend/src/components/PatientRegistration/, src/frontend/DoctorModuleApp.tsx, client/) versus frontend/apps/web/ and login-portal/.
- [ ] **Frontend/Backend Boundaries**: Verify that no frontend code in frontend/apps/web/ imports backend/Prisma modules directly, and check whether API contracts (frontend/apps/web/src/types/index.ts and login-portal/src/types/auth.ts) match backend DTOs/Zod schemas.
- [ ] **Service Dependencies**: Check how backend/api/src/modules/ai/ communicates with gemini_spark_mcp_server.py and gemini_web2api.py, and what happens when Python services are offline.
- [ ] **Configuration Inconsistencies**: Compare backend/api/config/env.ts, backend/api/config/database.ts, backend/prisma.config.ts, and .env.example files across backend/,

frontend/apps/web/, and login-portal/ for mismatched variable names, ports, or default fallbacks.
- [ ] **Error Handling & Logging**: Check whether Express 5 async error propagation is handled uniformly in error.middleware.ts / errorHandler.ts and whether morgan or console.log statements leak PII/tokens.
- [ ] **Circular Dependencies & Unnecessary Complexity**: Check cross-module imports in backend/api/src/modules/ (e.g., orders ↔ invoices ↔ payments ↔ patients) for circular dependencies.

B. Frontend (frontend/apps/web/ & login-portal/)

- [ ] **Page & Route Completeness**: Inspect all 24 route directories under frontend/apps/web/src/app/ and 5 pages in login-portal/src/pages/. Identify which pages are wired to live APIs via src/lib/api.ts versus rendering hardcoded/mock data (e.g., check realWorldDemoAnalyzers.ts usage).
- [ ] **Forms, Validation & Loading States**:
  - [ ] Check typo-named form components (DoctorFrom.tsx, OrderFrom.tsx, PatientFrom.tsx, TestCategoryFrom.tsx, TestFrom.tsx) vs wizard/modal forms (RegisterPatientWizard.tsx, MultiStepPatientForm.tsx, RegisterOrderWizard.tsx, RegisterDoctorWizard.tsx) to see which are active and whether client-side validation matches backend Zod schemas.
  - [ ] Verify loading skeletons (loading.tsx only exists in orders/ and patients/) and button disable states during form submissions.
- [ ] **Error & Empty States**: Check why error.tsx only exists in orders/ and patients/, and verify how API 4xx/5xx errors and empty table states are displayed across all modules.
- [ ] **API Integration**: Inspect frontend/apps/web/src/lib/api.ts, frontend/apps/web/src/hooks/use.Api.ts (note the dot in filename), and login-portal/src/lib/api.ts for base URL configuration, timeout handling, response envelope unwrapping, and 401 token refresh interceptors.
- [ ] **Authentication & Session Handling**: Reconcile the dual login implementation between login-portal/ (Port 5173) and frontend/apps/web/src/app/login/ (Port 3001). Inspect auth-storage.ts, AuthProvider.tsx, useAuth.ts, and webauthn.ts to verify how tokens are stored (localStorage vs httpOnly cookies) and cleared on logout.
- [ ] **Authorization-Aware UI**: Inspect ProtectedRoute.tsx, sidebar.tsx, and settingsAccess.ts to verify whether role restrictions (SUPER_ADMIN through BILLING_STAFF) hide unauthorized routes and actions.
- [ ] **Accessibility & Responsive Behavior**: Check modal focus trapping (Modal.tsx), keyboard navigation in SmartResultEntryGrid.tsx and CommandPalette.tsx, ARIA labels, and print stylesheet fidelity (PatientPrintTemplate.css, globals.css).
- [ ] **Broken Navigation & Dead Controls**: Check sidebar.tsx, header.tsx, QuickActions.tsx, and action dropdowns for links pointing to unimplemented routes or buttons with no onClick handlers.

C. Backend (backend/api/)

- [ ] **API Route Completeness**: Trace every router mounted in backend/api/src/app.ts. Verify whether all 28 documented modules (including receipts, whatsapp-advanced, and /api/communications/{logs,stats,sms/templates} noted in Risk #4) are mounted and reachable under /api/v1 and /api/*.
- [ ] **Input Validation**: Inspect *.validation.ts / *.validator.ts / *.schema.ts across all modules to confirm Zod validation middleware is applied to req.body, req.query, and req.params before controller execution.
- [ ] **Consistent Error Responses**: Verify that http-error.ts, response.ts, and the active error middleware return the documented { success: false, error, code } envelope without leaking stack traces or Prisma internal errors in production.
- [ ] **Authentication & Authorization Enforcement**: Audit every .routes.ts file to verify that JWT authentication and rbac.middleware.ts (permissions.ts) are enforced on all non-public endpoints, and

check whether any sensitive routes are accidentally left public.
- [ ] **Business-Rule Enforcement**: Verify server-side enforcement of state transitions (orders, samples, invoices, approvals, refunds), pathologist credential checks on sign-off, and single-open-session enforcement on cash counters.
- [ ] **Transactions & Concurrency**: Audit multi-table writes (POST /api/patients/with-order, order creation + package expansion + invoice generation, order cancellation + invoice void + payment reversal, EMPI merge, payment recording + wallet/cash-counter update, pathologist sign-off + report creation) for atomic prisma.$transaction usage.
- [ ] **Rate Limiting**: Inspect security.middleware.ts and app.ts to verify express-rate-limit configuration on /api/auth/*, OTP endpoints, public report verification (/verify-report/:id), and webhook routes.
- [ ] **Idempotency**: Inspect payment.service.ts, order.service.ts, and abdm webhook handlers. Evaluate the documented in-memory idempotency store (x-idempotency-key) for memory leaks, race conditions, and multi-process failure.
- [ ] **Safe File Uploads & Report Generation**: Audit doctor signature uploads, document vault uploads (doctor_documents), lab logo uploads, and report.pdf.service.ts (Puppeteer) for path traversal, SSRF, malicious HTML injection in PDF templates, unrestricted file types/sizes, and zombie Chromium processes.
- [ ] **Logging Without Exposing Sensitive Data**: Verify that request loggers, error logs, and communication_logs scrub passwords, JWTs, OTPs, TOTP secrets, and government IDs (Aadhaar/PAN/ABHA).

D. Database (backend/prisma/)

- [ ] **Schema Consistency**: Inspect backend/prisma/schema.prisma to verify the exact model count, naming conventions (@map / @@map), enum definitions, and data types (especially Decimal vs Float for financial amounts and lab results).
- [ ] **Relations, Foreign Keys, Unique Constraints & Indexes**: Verify foreign key onDelete behaviors (Restrict, Cascade, SetNull), unique constraints on uhid, orderNumber, barcode, invoiceNumber, and composite indexes for high-frequency queries.
- [ ] **Prisma Schema & Migration Alignment**: Compare the 17 migration SQL files in backend/prisma/migrations/ against schema.prisma. Investigate Risk #2 (accounting_sequences drift) and check why 20260924000000_add_missing_patient_columns and 20261010040000_add_missing_model_columns were needed.
- [ ] **Transaction Safety & Race Conditions**: Inspect backend/api/src/services/sequence.service.ts and UHID/Order/Sample/Invoice sequence generators to verify whether concurrent requests can generate duplicate sequence numbers (UHID-YYYY-XXXXX, ORD-YYYYMMDD-XXXXX, SMP-YYYYMMDD-XXXXX, INV-YYYYMMDD-XXXXX).
- [ ] **Data Integrity & Soft-Delete Behavior**: Check how DELETE /api/patients/:id and other delete operations implement soft-delete (deletedAt / isActive), and whether list/search queries and EMPI duplicate checks filter out soft-deleted records.
- [ ] **Safe Migration & Rollback Requirements**: Inspect backend/manual_migration_guide.md, backend/add-missing-column.js, backend/scripts/_apply_sql.js, and backend/scripts/push-schema.ts to identify any unversioned schema changes.
- [ ] **Tenant / Branch Isolation**: Section 11.2 and 5.5 mention multi-branch configuration. Verify whether branchId / tenantId exists on clinical and financial models in schema.prisma and whether queries enforce branch isolation.

E. Laboratory Workflows

- [ ] **Patient Registration, Search & EMPI**: Verify UHID generation, draft registration (patient.draft.service.ts), phonetic/probabilistic duplicate detection (patient.empi.service.ts), and whether POST /api/patients/empi/merge safely repoints all child records (orders, samples, invoices,

payments, patient_advances, patient_consents, abdm_registrations) within a transaction.
- [ ] **Doctor & Referral Mapping**: Verify referring doctor assignment on orders, commission calculation (PERCENTAGE vs FLAT_AMOUNT), TDS calculation, and payout voucher generation in doctor.commission.ts.
- [ ] **Test Catalog & Reference Ranges**: Verify LOINC code storage, parameter unit/decimal rules, age-in-days/years and gender boundary matching in reference_ranges (ensuring no gaps between newborn/infant/pediatric/adult/senior brackets), and package-to-test expansion.
- [ ] **Order Creation & Cancellation**: Verify duplicate test prevention within an order, package expansion pricing, priority flags (URGENT, STAT), and atomic cancellation (POST /api/orders/:id/cancel) when samples are already collected or results entered.
- [ ] **Sample Accession, Barcode Tracking & Rejection**: Verify the 7-stage state machine (REGISTERED → COLLECTED → RECEIVED → PROCESSING → RESULTS_ENTERED → APPROVED → DISPATCHED), Code 128 Mod-103 checksum calculation in barcode.service.ts, vacutainer tube mapping, and sample rejection/redraw workflows (SampleRejectModal.tsx, SampleRedrawRegister.tsx).
- [ ] **Result Entry, Validation & Flags**: Verify numeric/qualitative result storage, auto reference-range comparison, LOW/HIGH (ABNORMAL) and LL/HH (CRITICAL panic) flagging, 72-hour delta-check math, and Westgard QC rule evaluation (POST /api/results/evaluate-qc).
- [ ] **Critical-Result Notification & Acknowledgment**: Verify Socket.io panic_alert_new emission, SMS/WhatsApp escalation, and whether pathologist sign-off is blocked until CriticalValueAcknowledgment is recorded.
- [ ] **Pathologist Approval, Report Release & Addendums**: Verify role/credential enforcement on POST /api/approvals/sign-off, pre-sign-off ResultAmendment logging, post-sign-off immutability of results, ReportAddendum versioning, and public QR verification (/verify-report/:id).

F. Billing and Business Operations

- [ ] **Test Pricing, Discounts & Tax Calculations**: Inspect money.ts and invoice.service.ts to verify Indian banker's rounding, tax-inclusive vs tax-exclusive math, intra-state (CGST 9% + SGST 9%) vs inter-state (IGST 18%) determination, zero-rated exemptions, and discount authorization limits.
- [ ] **Invoice Creation & Numbering**: Verify INV-YYYYMMDD-XXXXX sequence atomicity and prevent duplicate invoice generation for a single order.
- [ ] **Payments, Partial Payments, Balances, Wallet & Refunds**: Verify partial payment state transitions (CONFIRMED → PARTIALLY_PAID → PAID), patient_advances wallet credit/debit ledger integrity, B2B settlements, receivables aging calculations, and refunds approval chain + invoice voiding.
- [ ] **Cash Counter Sessions**: Verify single-open-session enforcement per user, cash movement logging, and shift close expected-vs-actual reconciliation math in cash-counter.service.ts.
- [ ] **Barcode & Invoice Printing**: Verify thermal label (50×25mm) and ESC/POS receipt (80mm) generation in barcode.service.ts and receipt.service.ts.

G. Analyzer and Interoperability

- [ ] **ASTM/HL7 Parsing & Validation**: Inspect analyzer.service.ts to determine how ASTM E1381/E1394 (H|P|O|R|L frames, checksums, escape characters) and HL7 v2.5 messages are ingested, parsed, and validated.
- [ ] **Instrument-to-Test & Order Matching**: Verify how analyzer_test_mappings resolves instrument channel codes to test_parameters, and how incoming barcode/sample IDs match active samples and orders (including unmatched result handling for UnmatchedResultsQueue.tsx).
- [ ] **Duplicate Messages, ACK/NAK, Retries & Raw Retention**: Check whether raw ASTM/HL7 payloads are persisted for audit/replay, whether duplicate frames overwrite approved results, and how ACK/NAK responses and communication errors (CommunicationErrorLog.tsx) are handled.

- [ ] **ABDM / FHIR Interoperability**: Inspect backend/api/src/modules/abdm/ (abha.service.ts, hip.service.ts, fhir.service.ts, consent.service.ts, crypto.service.ts, data-transfer.service.ts) to verify M1/M2/M3 compliance, FHIR R4 DiagnosticReport bundle validity, and webhook RSA SPKI verification.

H. Security and Privacy

- [ ] **Authentication & Session Security**: Verify HMAC-SHA256 pepper + bcryptjs implementation in utils/password.ts, check all provisioning/reset scripts in backend/scripts/ and backend/reset-password.js for hash compatibility (Risk #3), verify JWT secret entropy validation, refresh token rotation/revocation in user_sessions, and TOTP brute-force lockout.
- [ ] **WebSocket Security**: Inspect backend/api/src/lib/websocket.ts to verify whether Socket.io handshakes authenticate JWT tokens and restrict room subscriptions by role/branch (Risk #8).
- [ ] **IDOR & Cross-Role Access**: Check whether object lookup endpoints (/api/patients/:id, /api/orders/:id, /api/reports/:id/pdf, /api/invoices/:id) verify user permissions and branch ownership.
- [ ] **Secrets Management & Encryption at Rest**: Inspect encryption.ts and field-crypto.ts. Verify whether totpSecretEnc, government ID tokens, and sensitive patient fields are encrypted using AES-256-GCM at rest (Risk #6), and ensure no real API keys or credentials exist in tracked .env or script files.
- [ ] **DPDP Act 2023 & Sensitive Identifier Redaction**: Verify granular consent enforcement (patient_consents), PII masking in patient.mapper.ts and compliance.ts, and strict redaction of government identifiers in API responses, logs, and public report verification endpoints.
- [ ] **Audit Log Integrity**: Inspect audit.service.ts to verify whether hash chaining is actually implemented and whether audit logs are protected against UPDATE/DELETE via API or DB roles.

I. Reliability and Performance

- [ ] **Database Connection Pooling**: Inspect backend/api/config/database.ts and backend/api/src/lib/prisma.ts to verify single PrismaClient instantiation, pg Pool max / timeout settings (Risk #7), and connection cleanup.
- [ ] **Slow Queries, N+1 & Pagination**: Check dashboard analytics (dashboard.service.ts, order.analytics.ts), worklists (samples, results, approvals), and EMPI search for unbounded queries or N+1 loops.
- [ ] **Resource Leaks & Timeouts**: Check Puppeteer browser instance lifecycle in report.pdf.service.ts, in-memory cache/idempotency eviction in utils/cache.ts, and httpx/gateway timeouts in ABDM, WhatsApp, and Python Gemini bridges.
- [ ] **Backup, Restore & Graceful Shutdown**: Inspect backup.service.ts and utils/backup.ts for pg_dump command injection risks, backup file access controls, restore verification, and SIGTERM/SIGINT handling in server.ts.

J. Testing and Delivery

- [ ] **Existing Tests & Scripts**: Inspect test/aiStudio.test.js, PixelPerfectInvoice.test.tsx, backend/test-*.js|ts, and backend/scripts/test-*.js|ts to determine what test runner (if any) is configured and whether tests mutate the live database.
- [ ] **Test Data Safety**: Audit backend/scripts/ (wipe-to-single-admin.ts, reset-all-data.ts, delete-admin.js, delete-user.ts, seed-data.ts) for environment guards preventing execution against production databases.
- [ ] **Type Checks, Linting & Production Build**: Check tsconfig.json in backend/, frontend/apps/web/, and login-portal/, inspect frontend/apps/web/lint-out.txt and backend/README-TYPESCRIPT-FIX.md for known compilation/lint errors, and inspect frontend/apps/web/scripts/build.mjs.

4. Critical Workflow Test Matrix

The repository auditor must trace and verify these 10 core clinical and financial workflows across the frontend, backend, and database layers:

Workflow ID	Clinical / Business Workflow	Frontend Entry Point(s) ([cite: 2])	Backend Route & Service ([cite: 1, 2])	Prisma Models Affected ([cite: 1])	Key Verification & Invariant Checks (REQUIRES REPOSITORY VERIFICATION)
WF-01	**Authentication, TOTP MFA & Session Refresh**	login-portal/src/pages/{LoginPage,MfaSetupPage,MfaVerifyPage}.tsx AND frontend/apps/web/src/app/{login,mfa-setup,mfa-verify}/	POST /api/auth/{login,mfa/setup,mfa/verify,refresh,logout} (modules/auth/auth.service.ts)	users, user_sessions, mfa_backup_codes, login_alerts, password_history	• Verify HMAC-SHA256 pepper + bcrypt cost 12. • Verify 5-fail brute-force lockout (lockedUntil). • Confirm whether login-portal or apps/web/src/app/login is canonical.
WF-02	**Patient Registration, DPDP Consent & EMPI Merge**	frontend/apps/web/src/app/patients/{new,[id]}/ (RegisterPatientWizard.tsx,	POST /api/patients, POST /api/patients/empi/{search,merge} (patient.servi	patients, patient_consents, patient_history_entries, audit_logs	• Verify UHID-YYYY-XXXXX concurrency safety.

Workflow ID	Clinical / Business Workflow	Frontend Entry Point(s) ([cite: 2])	Backend Route & Service ([cite: 1, 2])	Prisma Models Affected ([cite: 1])	Key Verification & Invariant Checks (REQUIRES REPOSITORY VERIFICATION)
		MultiStepPatientForm.tsx)	ce.ts, patient.empi.service.ts)		• Verify duplicate detection scoring. • Verify EMPI merge atomically updates all foreign keys without data loss.
WF-03	**Order Creation, Package Expansion & Atomic Cancel**	frontend/apps/web/src/app/orders/new/page.tsx (RegisterOrderWizard.tsx)	POST /api/orders, POST /api/patients/with-order, POST /api/orders/:id/cancel (order.service.ts)	orders, order_tests, test_packages, invoices, invoice_items, payments	• Verify package constituent test expansion & deduplication. • Verify POST /api/orders/:id/cancel atomically voids invoices/payments and blocks cancel if report is already

Workflow ID	Clinical / Business Workflow	Frontend Entry Point(s) ([cite: 2])	Backend Route & Service ([cite: 1, 2])	Prisma Models Affected ([cite: 1])	Key Verification & Invariant Checks (REQUIRES REPOSITORY VERIFICATION)
					dispatched.
WF-04	**POS GST Billing, Idempotent Payment & Cash Counter**	frontend/apps/web/src/app/invoices/ & /payments (QuickPOSBilling.tsx, CollectPaymentModal.tsx, ShiftHandoverModal.tsx)	POST /api/invoices, POST /api/payments, POST /api/cash-counter/{open,close,movement}	invoices, payments, payment_audit_logs, cash_counters, cash_counter_sessions, cash_movements, patient_advances	• Verify CGST/SGST vs IGST & banker's rounding. • Verify x-idempotency-key behavior under concurrent duplicate POSTs. • Verify single open cash session per user & shift reconciliation math.
WF-05	**Phlebotomy Accession, Code 128 Barcode & State Machine**	frontend/apps/web/src/app/samples/collect/ & /samples/[id]/label (SpecimenAccessioningM	POST /api/samples, PATCH /api/samples/:id/status, GET /api/barcodes/label/:id	samples, sample_tracking_history, orders	• Verify SMP-YYYYMMDD-XXXXX uniqueness & Code 128 Mod-103 checksum.

Workflow ID	Clinical / Business Workflow	Frontend Entry Point(s) ([cite: 2])	Backend Route & Service ([cite: 1, 2])	Prisma Models Affected ([cite: 1])	Key Verification & Invariant Checks (REQUIRES REPOSITORY VERIFICATION)
		odal.tsx, SampleLabelPrint.tsx)	(sample.service.ts, barcode.service.ts)		• Verify strict enforcement of the 7-stage state machine (preventing illegal backward or skipped jumps).
WF-06	**Analyzer ASTM/HL7 Ingestion & Westgard QC / Delta Check**	frontend/apps/web/src/app/analyzers/ (LiveResultFeed.tsx, UnmatchedResultsQueue.tsx, ShiftQCGate.tsx)	/api/analyzers/*, POST /api/results/evaluate-qc (analyzer.service.ts, result.service.ts)	analyzers, analyzer_test_mappings, qc_runs, qc_results, results	• Verify ASTM `H
WF-07	**Manual Result Entry, Panic Alerts & Critical ACK**	frontend/apps/web/src/app/results/ (SmartResultEntryGrid.tsx, CriticalPanicEscalationHub.tsx)	POST /api/results (result.service.ts, lib/websocket.ts)	results, critical_value_acknowledgments, samples	• Verify age/gender reference range selection. • Verify HH/LL critical flag triggers Socket.io panic_alert_new and

Workflow ID	Clinical / Business Workflow	Frontend Entry Point(s) ([cite: 2])	Backend Route & Service ([cite: 1, 2])	Prisma Models Affected ([cite: 1])	Key Verification & Invariant Checks (REQUIRES REPOSITORY VERIFICATION)
					requires CriticalValue Acknowledgment before approval.
WF-08	**Pathologist Review, Amendment & Digital Sign-Off**	frontend/apps/web/src/app/approvals/ (DoctorWorkstationView.tsx, ApprovalReviewModal.tsx)	GET /api/approvals/queue, POST /api/approvals/sign-off, POST /api/approvals/:id/reject (approval.service.ts)	approvals, result_amendments, results, samples, reports, audit_logs	• Verify only PATHOLOGIST (or authorized credentialed role) can sign off. • Verify pre-sign-off edits create ResultAmendment records. • Verify Phase 4.2/4.3 completion status vs roadmap (40%).
WF-09	**NABL PDF Report Generation, Addendum & QR**	frontend/apps/web/src/app/reports/ & /verify-report/[reportId]	GET /api/reports/:id/pdf, POST /api/reports/:id/share,	reports, report_share_links, report_adde	• Verify whether Puppeteer server-side PDF

Workflow ID	Clinical / Business Workflow	Frontend Entry Point(s) ([cite: 2])	Backend Route & Service ([cite: 1, 2])	Prisma Models Affected ([cite: 1])	Key Verification & Invariant Checks (REQUIRES REPOSITORY VERIFICATION)
	Verification	(PremiumLabReport.tsx, ReportAddendumModal.tsx)	POST /api/reports/:id/addendum, GET /verify-report/:id	ndums	(report.pdf.service.ts) is functional (Roadmap says Phase 5 is 0%). • Verify addendum versioning and tamper-proof QR verification.
WF-10	**Doctor Commission Accrual & Payout Settlement**	frontend/apps/web/src/app/doctors/ (DoctorCommissionPayoutModal.tsx, DoctorReferralBilling.tsx)	/api/doctors/* (doctor.service.ts, doctor.commission.ts)	doctors, doctor_organizations, doctor_commission_entries, doctor_payouts	• Verify commission accrual on order/invoice confirmation, reversal on order cancel/refund, and TDS deduction on payout.

5. Security and Privacy Audit Matrix

Security / Privacy Domain	Documented Control or Known Risk ([cite: 1])	Target Files to Inspect ([cite: 2])	Verification Method (REQUIRES REPOSITORY VERIFICATION)
Password Hashing & Provisioning Scripts	HMAC-SHA256(password, PASSWORD_PEPPER) + bcryptjs (cost 12); Risk #3 warns provisioning scripts may not align with utils/password.ts.	backend/api/src/utils/password.ts, backend/api/src/utils/password-policy.ts, backend/reset-password.js, `backend/scripts/*admin*.ts	js, backend/scripts/create-users.ts`
JWT, Sessions & TOTP MFA	15m Access Token, 7d Refresh Token stored in user_sessions, TOTP MFA with encrypted secret (totpSecretEnc) and 10 backup codes.	backend/api/src/utils/{jwt,totp}.ts, backend/api/src/modules/auth/auth.service.ts, backend/api/middleware/auth*.ts	Check JWT algorithm pinning, secret fallback defaults in env.ts, refresh token rotation/revocation on logout/password reset, and whether totpSecretEnc is actually encrypted before DB insert.
RBAC & Endpoint Authorization	8-tier role hierarchy (SUPER_ADMIN to BILLING_STAFF) + role_permissions table.	backend/api/middleware/rbac.middleware.ts, backend/api/src/lib/permissions.ts, all *.routes.ts in backend/api/src/modules/	Build a route-by-route permission matrix from *.routes.ts. Flag any route missing auth/RBAC middleware or allowing low-privilege roles (RECEPTIONIST, PHLEBOTOMIST) to approve results or issue refunds.
PII & Government ID Protection (DPDP Act 2023)	Granular consent (patient_consents), PII masking, field encryption (aadhaarReferenceToken); Risk #6 warns government IDs are stored in plaintext in	backend/prisma/schema.prisma, backend/api/src/utils/{encryption,field-crypto,compliance}.ts, backend/api/src/modules/patients/patient.	Verify AES-256 encryption on all government ID fields in schema.prisma and patient.service.ts. Ensure government IDs are never echoed

Security / Privacy Domain	Documented Control or Known Risk ([cite: 1])	Target Files to Inspect ([cite: 2])	Verification Method (REQUIRES REPOSITORY VERIFICATION)
	some fields.	mapper.ts	in plaintext in API responses, logs, or error payloads.
WebSocket Authentication	Risk #8 states WebSocket auth is undocumented and Socket.io connections should verify JWT on handshake.	backend/api/src/lib/websocket.ts, backend/api/src/server.ts	Check whether io.use() middleware validates Bearer JWTs on connection and prevents unauthenticated clients from listening to panic_alert_new or result_entered events.
Public Endpoints & Share Links (IDOR / Data Leak)	/verify-report/:id and report_share_links provide unauthenticated report verification/access.	backend/api/src/modules/reports/report.{routes,controller,service}.ts, frontend/apps/web/src/app/verify-report/[reportId]/page.tsx	Verify that /verify-report/:id uses high-entropy cryptographic tokens (not sequential IDs), enforces link expiry, and masks sensitive PII.
Webhook Authenticity & Replay Defense	ABDM webhooks (/v0.5/*) use RSA SPKI signature verification and ±5min timestamp validation; WhatsApp webhooks also exist.	backend/api/src/modules/abdm/{abdm.middleware,crypto.service}.ts, backend/api/src/modules/communications/whatsapp-webhook.controller.ts	Confirm cryptographic signature verification is enforced (not bypassed in dev/prod) on both ABDM /v0.5/* and WhatsApp webhook endpoints.
Audit Log Tamper-Evident Chaining	audit_logs records actor, IP, before/after JSON delta, and uses hash chaining for 7-year NABL compliance.	backend/api/src/modules/audit/audit.{service,controller,routes}.ts, backend/prisma/schema.prisma	Verify whether hash chaining (previousHash + payload hash) is actually computed on insert and whether any API endpoint allows

Security / Privacy Domain	Documented Control or Known Risk ([cite: 1])	Target Files to Inspect ([cite: 2])	Verification Method (REQUIRES REPOSITORY VERIFICATION)
			modifying or deleting audit logs.
Backup & Command Execution Safety	backup module triggers on-demand pg_dump and allows backup download/deletion.	backend/api/src/modules/backup/backup.{routes,controller,service}.ts, backend/api/src/utils/backup.ts	Verify SUPER_ADMIN-only access, check child_process calls for shell injection via filenames/DB URLs, and check path traversal on backup download/delete routes.

6. Database and Migration Audit Matrix

Audit Area	Specific Artifact / Target ([cite: 1, 2])	Risk / Evidence in Supplied Docs ([cite: 1, 2])	Required Verification Check (REQUIRES REPOSITORY VERIFICATION)
Schema-to-Migration Drift	backend/prisma/schema.prisma vs 17 folders in backend/prisma/migrations/ (20260825092845_init through 20261014050000_enterprise_empi_module).	Risk #2 flags accounting_sequences drift; migration history includes 20260831055935_sync_schema, 20260924000000_add_missing_patient_columns, 20261010040000_add_missing_model_columns, and 20261013120000_add_accounting_sequences_prefix.	Compare all tables/columns/enums created across the 17 migration.sql files against schema.prisma. Identify any model or column present in schema.prisma that is missing from migrations/.
Out-of-Band Schema Mutation	backend/add-missing-column.js,	Presence of manual ALTER TABLE and db	Inspect every script to see what

Audit Area	Specific Artifact / Target ([cite: 1, 2])	Risk / Evidence in Supplied Docs ([cite: 1, 2])	Required Verification Check (REQUIRES REPOSITORY VERIFICATION)
Scripts	backend/check-schema.js, backend/test-column-names.js, backend/scripts/_apply_sql.js, backend/scripts/execute-sql.js, backend/scripts/push-schema.ts.	push scripts indicates historical schema drift outside Prisma Migrate.	columns/tables they patch and whether those changes have been folded into the official Prisma migrations.
Sequence Generation & Concurrency	backend/api/src/services/sequence.service.ts and accounting_sequences table.	Generates UHID-YYYY-XXXXX, ORD-YYYYMMDD-XXXXX, SMP-YYYYMMDD-XXXXX, INV-YYYYMMDD-XXXXX.	Verify whether sequence increments use atomic database operations (UPDATE ... RETURNING or INSERT ... ON CONFLICT DO UPDATE) inside transactions, or unsafe SELECT max() + 1 reads prone to race conditions.
Financial & Clinical Precision	invoices, invoice_items, payments, refunds, patient_advances, doctor_commission_entries, results, reference_ranges.	Migration 20260929120000_widen_order_gst_amount indicates past numeric overflow/precision issues.	Verify all monetary columns use Decimal with sufficient precision/scale (e.g., @db.Decimal(12, 2)) and never IEEE-754 Float.
Referential Integrity & Cascade Rules	All 81 documented models in backend/prisma/schema.prisma.	Clinical & financial records must be retained for 7 years (NABL/GST).	Ensure onDelete: Cascade is NOT set on patients → orders/invoices/reports or users → audit_logs/approvals.

Audit Area	Specific Artifact / Target ([cite: 1, 2])	Risk / Evidence in Supplied Docs ([cite: 1, 2])	Required Verification Check (REQUIRES REPOSITORY VERIFICATION)
Index Coverage	Section 6.2 documents 9 critical indexes (users, patients, orders, samples, results, audit_logs, invoices).	Missing indexes on foreign keys or filter columns cause full-table scans on worklists and analytics.	Verify all @@index and @unique annotations in schema.prisma against actual WHERE and ORDER BY clauses in *.service.ts.
Destructive Script Safeguards	backend/scripts/wipe-to-single-admin.ts, backend/scripts/reset-all-data.ts, backend/scripts/delete-admin.js, backend/scripts/delete-user.ts.	High risk of accidental data destruction if executed in staging/production.	Inspect scripts to check if they block execution when NODE_ENV === 'production' and what tables they truncate/delete.

7. Analyzer and Integration Audit Matrix

Integration / Interface	Documented Capability & Protocol ([cite: 1])	Target Repository Files ([cite: 2])	Required Verification & Edge-Case Audit (REQUIRES REPOSITORY VERIFICATION)
ASTM E1381/E1394 & HL7 v2.5 Analyzer Interface	Bidirectional ASTM (H|P|O|R|L frames, \r\n delimited) and HL7 v2.5 Host-Query; Westgard QC (1-3s, 2-2s, 4-1s, 10x, R-4s); 72h delta-check.	backend/api/src/modules/analyzers/analyzer.{controller,routes,service,validation}.ts, frontend/apps/web/src/components/analyzers/{VirtualAnalyzerSimulator,LiveResultFeed,UnmatchedResultsQueue,CommunicationErrorLog}.tsx	• Verify whether TCP/Serial socket listeners exist or if ingestion is HTTP REST only. • Check framing (ENQ/ACK/NAK/STX/ETX/EOT), Modulo-256

Integration / Interface	Documented Capability & Protocol ([cite: 1])	Target Repository Files ([cite: 2])	Required Verification & Edge-Case Audit (REQUIRES REPOSITORY VERIFICATION)
			checksum validation, and malformed payload protection. • Check handling of unmatched barcodes, duplicate result frames, and locked/approved samples.
ABDM / ABHA v0.5 (M1, M2, M3)	M1 Aadhaar OTP + ABHA linking; M2 HIP Care Context; M3 FHIR R4 DiagnosticReport bundle; /v0.5/* RSA SPKI webhooks. Roadmap marks Phase 6 as **50% partial**.	backend/api/src/modules/abdm/{abdm.config,abdm.controller,abdm.gateway.client,abdm.middleware,abdm.routes,abdm.validation,abha.service,consent.service,crypto.service,data-transfer.service,fhir.service,hip.service}.ts	• Check which M1/M2/M3 methods call real ABDM Gateway endpoints vs returning mock data. • Validate FHIR R4 DiagnosticReport structure and LOINC coding. • Verify Fidelius/ECDH encryption in crypto.service.ts and data-transfer.service.ts.
WhatsApp Cloud API, SMS & Email	Meta Cloud API templates/campaigns/chatbot, SMS gateway,	backend/api/src/lib/{communication-providers,whatsapp-ai.service}.ts,	• Verify which routes in communication.routes.ts and

Integration / Interface	Documented Capability & Protocol ([cite: 1])	Target Repository Files ([cite: 2])	Required Verification & Edge-Case Audit (REQUIRES REPOSITORY VERIFICATION)
	Nodemailer/Gmail SMTP. Roadmap marks Phase 7 as **40% partial**; Risk #4 notes /api/communications/{logs,stats,sms/templates} 404.	backend/api/src/utils/gmail.ts, backend/api/src/modules/communications/*, backend/scripts/process-scheduled-messages.ts	whatsapp-advanced.routes.ts are implemented vs missing (Risk #4). • Check how process-scheduled-messages.ts is scheduled/invoked. • Check retry/backoff logic and failure logging in communication_logs.
Gemini AI Clinical Studio & MCP Bridges	FastAPI bridges (gemini_spark_mcp_server.py on :8000, gemini_web2api.py on :8001) + backend ai module. Roadmap marks Phase 8 as **40% partial**.	gemini_spark_mcp_server.py, gemini_web2api.py, model_capabilities.yaml, backend/api/src/modules/ai/{ai.controller,ai.routes,ai.service,ai.types,gemini.service}.ts	• Verify whether modules/ai/ calls Google Gemini directly or routes through the Python FastAPI proxies. • Check whether patient PII is stripped/anonymized before sending clinical data to external LLM APIs (DPDP compliance).
Thermal Barcode & ESC/POS Receipt	Code 128 Mod-103 (50×25mm ZPL/PNG) and ESC/POS	backend/api/src/modules/barcode/*, backend/api/src/mod	• Check why backend/api/src/modules/receipts/ has no

Integration / Interface	Documented Capability & Protocol ([cite: 1])	Target Repository Files ([cite: 2])	Required Verification & Edge-Case Audit (REQUIRES REPOSITORY VERIFICATION)
Printing	(80mm) thermal receipt generation.	ules/receipts/*, frontend/apps/web/src/components/invoices/ThermalReceipt.tsx, frontend/apps/web/src/components/samples/SampleLabelPrint.tsx	receipt.routes.ts in the file listing and determine where receipt.controller.ts is mounted. • Verify Code 128 Mod-103 checksum accuracy.

8. Testing and Deployment Audit Matrix

Audit Area	Repository Artifacts ([cite: 1, 2])	Key Verification Questions (REQUIRES REPOSITORY VERIFICATION)
Unit, Integration & E2E Test Suites	• test/aiStudio.test.js • frontend/apps/web/src/components/invoices/PixelPerfectInvoice.test.tsx • `backend/test-{all-endpoints,api-auth,column-names,dashboard,db,order-analytics,patient-creation,patient-validation,patients-api,patients,phase2-billing,phase3-analytics,phase3-auth,tables,tests-qu	ts • backend/scripts/test-{auth-complete,empi-capabilities,gmail-recovery,login,phase1-audit}.js

Audit Area	Repository Artifacts ([cite: 1, 2])	Key Verification Questions (REQUIRES REPOSITORY VERIFICATION)
	ery}.js	
TypeScript Compilation & Linting	• backend/tsconfig.json, backend/README-TYPESCRIPT-FIX.md • frontend/apps/web/tsconfig.json, frontend/apps/web/lint-out.txt • login-portal/tsconfig*.json, login-portal/.oxlintrc.json	• What existing lint/type errors are recorded in frontend/apps/web/lint-out.txt and backend/README-TYPESCRIPT-FIX.md? • Does npx tsc --noEmit pass cleanly in backend/, frontend/apps/web/, and login-portal/? • Does backend rely on --transpile-only (noted in Risk #5) to mask TypeScript compilation errors?
Build & Packaging	• frontend/apps/web/scripts/build.mjs, frontend/apps/web/next.config.js • login-portal/vite.config.ts • requirements.txt, start_gemini_spark_mcp.bat	• What custom logic is inside frontend/apps/web/scripts/build.mjs? • Does next.config.js ignore TypeScript (ignoreBuildErrors) or ESLint (ignoreDuringBuilds) errors during production builds?
Environment & Deployment Readiness	• DEPLOYMENT_GUIDE.md, START_SERVER_FIX.md	sh`

Audit Area	Repository Artifacts ([cite: 1, 2])	Key Verification Questions (REQUIRES REPOSITORY VERIFICATION)
	• backend/.env.example, frontend/apps/web/.env.example, login-portal/.env.example • `scripts/validate-*.ps1	

9. Issue Classification Rules

All findings discovered during repository inspection must be classified using both the **4-Tier Priority Scale (P0–P3)** and the **6-Category Finding Taxonomy**. Severity and priority must be assigned strictly based on **verified code evidence and actual clinical/financial/security impact**, never on speculation.

9.1 Priority Definitions (P0–P3)

Priority	Classification Name	Qualifying Criteria (Clinical, Financial, Security, Operational)
P0	**Critical Blocker**	• **Patient Safety**: Incorrect reference range flagging, missed HH/LL panic alerts, result/patient mix-up (e.g., unsafe EMPI merge or ASTM mapping), unauthorized result modification after sign-off, or broken critical-value acknowledgment. • **Security & Privacy**: Unauthenticated access to clinical/financial endpoints, broken RBAC allowing

Priority	Classification Name	Qualifying Criteria (Clinical, Financial, Security, Operational)
		non-pathologists to sign off reports, plaintext government ID exposure, SQL/command injection (e.g., in backup), or authentication bypass. • **Data Loss / Financial Corruption**: Non-transactional payment/invoice mutations causing lost payments, race conditions generating duplicate UHIDs/barcodes/invoice numbers, or unguarded destructive scripts.
P1	**Major Defect / High-Impact Failure**	• Core workflow failure blocking patient registration, order creation, sample state transitions, result entry, pathologist approval, or PDF report generation. • Schema drift between schema.prisma and migrations/ causing runtime Prisma query crashes (e.g., Risk #2 accounting_sequences). • Inaccurate GST, discount, commission, or cash-counter reconciliation calculations.

Priority	Classification Name	Qualifying Criteria (Clinical, Financial, Security, Operational)
		• Unauthenticated WebSocket feeds broadcasting patient results/panic alerts (Risk #8).
P2	**Moderate Defect / Incomplete Feature / Reliability Gap**	• Documented sub-routes returning 404 (e.g., Risk #4 /api/communications/*) or roadmap Phase 4–8 features wired to mock/stubbed UI. • In-memory payment idempotency store losing state on restart or across cluster workers. • Missing database indexes on high-volume worklists, missing pagination bounds, or missing UI error/loading states. • Duplicate/shadowed controller and middleware files creating maintenance hazards.
P3	**Enhancement / Optimization / Low-Priority Improvement**	• Dead legacy files (public/, root index.js, typo-named *From.tsx components) that are not imported at runtime.

Priority	Classification Name	Qualifying Criteria (Clinical, Financial, Security, Operational)
		• Minor UI/UX polish, non-critical accessibility improvements, cold-start developer script optimizations (Risk #5), or documentation formatting updates.

9.2 Mandatory 6-Category Finding Taxonomy

Every logged issue must be explicitly tagged with exactly one of the following six verification categories:

1. **CONFIRMED_DEFECT**: The feature is implemented and wired, but code inspection or test execution proves it produces incorrect results, throws runtime errors, violates security/RBAC, or corrupts data.
2. **CONFIRMED_MISSING_FEATURE**: A capability required by a core workflow or claimed as complete in ARCHITECTURE.md (Phases 1–3) is completely absent from the codebase.
3. **INCOMPLETE_IMPLEMENTATION**: Partial code exists (e.g., UI component exists without backend route, or controller/service exists with TODO/mocked responses, such as Phases 4–10 roadmap items).
4. **POTENTIAL_RISK_REQUIRING_VERIFICATION**: Code exhibits a structural hazard (e.g., race condition under high concurrency, memory leak under load, or external gateway timeout) that requires live runtime or load testing to confirm.
5. **DOCUMENTATION_ONLY_CLAIM**: ARCHITECTURE.md describes an enterprise capability, compliance certification, or integration that has zero corresponding implementation in the repository.
6. **ENVIRONMENT_OR_DEPENDENCY_BLOCKER**: Inspection, build, or test execution is blocked by missing external credentials (e.g., ABDM, WhatsApp, Gemini API keys), OS-specific binaries (e.g., Puppeteer Chromium, PostgreSQL pg_dump), or broken package lockfiles.

10. Evidence Collection Template

The repository auditor must use the following 14-field template for **every** finding recorded during the audit. Findings lacking exact file paths, symbol names, or concrete evidence will be rejected.

```
### [ISSUE-ID] Short, Descriptive Title (e.g., [LAB-AUD-001] Unauthenticated
Socket.io Handshake Exposes Panic Alerts)

* **1. Unique Issue ID:** LAB-AUD-XXX
* **2. Title:** Concise summary of the defect, gap, or risk.
* **3. Severity & Priority:** [P0 | P1 | P2 | P3] — Category: [CONFIRMED_DEFECT
| CONFIRMED_MISSING_FEATURE | INCOMPLETE_IMPLEMENTATION |
POTENTIAL_RISK_REQUIRING_VERIFICATION | DOCUMENTATION_ONLY_CLAIM |
ENVIRONMENT_OR_DEPENDENCY_BLOCKER]
* **4. Affected Component:** [e.g., Backend Module: approvals | Frontend:
apps/web | Auth Portal | Prisma Schema | Analyzer Interface]
```

```
* **5. Exact File Path & Symbol:** `path/to/file.ts` → `ClassName.methodName()` or `functionName()` (Lines X-Y)
* **6. Evidence:** Direct code snippet, Prisma schema excerpt, route definition, or terminal/test output proving the finding (never include raw secrets or forbidden government ID digits).
* **7. Reproduction Steps:**
  1. Exact HTTP request / UI action / CLI command (non-destructive).
  2. Input payload or precondition state.
  3. Observed output.
* **8. Expected vs. Actual Behavior:**
  * *Expected:* What the clinical workflow, security policy, or `ARCHITECTURE.md` requires.
  * *Actual:* What the code actually executes or returns.
* **9. Patient, Business, Security, or Operational Impact:** Concrete impact on patient safety (e.g., delayed critical value alert), regulatory compliance (NABL, DPDP, ABDM, GST), financial integrity, or system availability.
* **10. Root Cause:** Underlying technical reason (e.g., missing `prisma.$transaction`, unmounted router in `app.ts`, missing RBAC middleware).
* **11. Recommended Fix:** Specific, minimal, non-breaking code or schema change required to remediate the issue while preserving existing working functionality.
* **12. Required Regression Test:** Unit, integration, or E2E test case specification to verify the fix and prevent regression.
* **13. Dependencies & Migration Risks:** Downstream impact on other modules, frontend API contracts, or Prisma database migrations (e.g., requires backfill or table lock).
* **14. Verification Status:** [UNVERIFIED | CONFIRMED_BY_STATIC_ANALYSIS | CONFIRMED_BY_RUNTIME_TEST | BLOCKED_BY_ENV]
```

11. Recommended Audit Execution Order

The next AI agent must execute the audit in the following **12-step non-destructive sequence**.

STRICT SAFETY GUARDRAIL: Do **NOT** execute destructive commands (prisma migrate reset, prisma db push --force-reset, wipe-to-single-admin.ts, reset-all-data.ts, DROP, TRUNCATE, or rm -rf). Require explicit human approval before modifying any source files, installing packages, running database migrations, or altering database state.

1. **Inspect Repository Structure & Legacy Footprint (Read-Only)**:
   - List and compare the active workspaces (backend/, frontend/apps/web/, login-portal/) against legacy/duplicate directories (index.js, src/, public/, client/, frontend/src/, backend/api/middleware/, backend/api/src/controllers|routes|schemas/).
   - Read .gsd/STATE.md, .gsd/SPEC.md, COMPLETE_UPGRADE_SUMMARY.md, PHASE_1_DEBUGGING_REPORT.md, PHASE_2_BILLING_AUDIT_REPORT.md, PHASE_3_ANALYTICS_AUDIT_REPORT.md, and PATIENT_MODULE_CLINICAL_LIS_AUDIT.md to understand prior audit context.
2. **Inspect Package Manifests, Lockfiles & Scripts (Read-Only)**:
   - Read package.json in root, backend/, frontend/, frontend/apps/web/, and login-portal/, plus requirements.txt.
   - Record all configured build, dev, lint, typecheck, and test scripts, and check dependency

versions against Section 10 of ARCHITECTURE.md.

3. **Inspect Environment Examples & Config Loaders Without Exposing Secrets (Read-Only)**:
    - Read backend/.env.example, frontend/apps/web/.env.example, login-portal/.env.example, backend/api/config/env.ts, backend/api/config/database.ts, and backend/prisma.config.ts.
    - **Never** print or log contents of live .env files. Verify whether required secrets (JWT_SECRET, JWT_REFRESH_SECRET, PASSWORD_PEPPER, encryption keys) fail fast on startup if missing or default to insecure hardcoded strings.
4. **Inspect Database Schema, Migrations & Sequence Logic (Read-Only)**:
    - Read backend/prisma/schema.prisma and all 17 migration.sql files under backend/prisma/migrations/.
    - Check for schema drift (specifically accounting_sequences and recent add_missing_* migrations), missing foreign key indexes, Float vs Decimal usage, and race conditions in backend/api/src/services/sequence.service.ts.
5. **Map Backend APIs to Frontend Pages & Components (Read-Only)**:
    - Read backend/api/src/server.ts, backend/api/src/app.ts, and all *.routes.ts files across backend/api/src/modules/ to build the authoritative table of mounted endpoints, auth/RBAC middleware, and Zod validators.
    - Cross-reference against frontend/apps/web/src/lib/api.ts, login-portal/src/lib/api.ts, and all route pages under frontend/apps/web/src/app/ to identify 404 dead links, unmounted routes (e.g., receipts, communications sub-routes), and mock-only UI components.
6. **Trace Critical Business, Clinical & Security Workflows (Read-Only)**:
    - Walk through Workflows **WF-01 through WF-10** (Section 4) and the Security Matrix (Section 5) in code.
    - Verify transaction boundaries (prisma.$transaction), state-machine guards (samples, orders, invoices, approvals), Westgard/delta-check math, panic alert acknowledgments, password pepper consistency across scripts, and PII encryption (field-crypto.ts).
7. **Run Safe, Non-Destructive Static Checks**:
    - Inspect existing diagnostic files (frontend/apps/web/lint-out.txt, backend/README-TYPESCRIPT-FIX.md).
    - Run read-only static analysis (npx tsc --noEmit in backend/, frontend/apps/web/, and login-portal/; npm run lint / oxlint; npx prisma validate in backend/) if node_modules are already installed.
8. **Evaluate & Run Existing Non-Destructive Unit Tests**:
    - Inspect test/aiStudio.test.js, PixelPerfectInvoice.test.tsx, and backend/test-*.js|ts **before** running them.
    - Only execute isolated unit tests that do not mutate a live database. Do not run integration scripts that insert/modify/wipe records in labcore_elis without explicit user approval and a verified isolated test database.
9. **Reproduce Suspected Defects Safely**:
    - For any suspected defects, verify via static code path tracing or read-only queries/isolated dry runs.
10. **Record Evidence Using the 14-Field Template**:
    - Document every finding with exact file paths, line numbers, function names, code excerpts, and clinical/financial/security impact (Section 10).
11. **Deduplicate & Cross-Link Root Causes**:
    - Consolidate duplicate symptoms (e.g., multiple endpoints failing due to a single unmounted router, shared middleware bug, or Prisma schema mismatch) into unified root-cause issues.
12. **Prepare the Prioritized Remediation Plan**:
    - Group all verified findings by **P0 → P1 → P2 → P3**, specify migration and dependency ordering, and present the remediation plan for human review before making any code changes.

12. Final Checklist for Antigravity IDE

Before concluding the repository audit in Antigravity IDE, the auditing agent must verify that every item below is satisfied:

- [ ] **Zero Unverified Assumptions**: No feature from NIKIL_LABCORE_ELIS_OS_COMPLETE_ARCHITECTURE.md has been marked operational without inspecting its actual .ts/.tsx implementation.
- [ ] **Zero Unauthorized Mutations**: No files were modified, no packages were installed, no migrations were applied, and no destructive scripts (wipe-to-single-admin.ts, reset-all-data.ts) were executed.
- [ ] **Strict Sensitive ID & Secret Redaction**: No raw credentials from .env files and no numeric digits of forbidden government identifiers (Aadhaar, MyNumber, RRN) appear anywhere in the audit report or evidence snippets.
- [ ] **Dual-Stack & Duplicate Code Resolved in Map**:
    - [ ] Root index.js, src/, and public/ vs backend/api/ status confirmed.
    - [ ] backend/api/middleware/ vs backend/api/src/ middleware status confirmed.
    - [ ] backend/api/src/controllers|routes|schemas/ vs backend/api/src/modules/ status confirmed.
    - [ ] login-portal/ (Port 5173) vs frontend/apps/web/src/app/login/ (Port 3001) auth flow ownership confirmed.
    - [ ] Active invoice template identified among the 6+ components in frontend/apps/web/src/components/invoices/.
- [ ] **All 8 Known Technical Debt Risks Checked**: All 8 risks from Section 15 of ARCHITECTURE.md (Dual server, accounting_sequences drift, script password hash alignment, /api/communications/* 404s, ts-node cold start, missing PII encryption at rest, DB connection pool limits, and WebSocket handshake auth) have been verified in code with exact file/line evidence.
- [ ] **Phase 4–10 Roadmap vs Code Reality Reconciled**: Actual implementation status of Pathologist Sign-Off (Phase 4), Puppeteer NABL PDF (report.pdf.service.ts, Phase 5), ABDM v0.5 (modules/abdm/, Phase 6), WhatsApp (modules/communications/, Phase 7), and AI Studio (modules/ai/ & Python bridges, Phase 8) is documented with evidence.
- [ ] **Complete Prioritized Issue Register Delivered**: Every finding includes all 14 required evidence fields (Section 10), a P0–P3 priority, and a 6-category classification tag (Section 9).
